How JS Works Under the Hood ?

What is JS?

It is a versatile, dynamically typed language that brings life to webpages by making them interactive. It is a synchronous and single-threaded programming language.

- Dynamically typed → Don’t need to assign a data type; it is determined at runtime.
- Single-threaded → It has one call stack and executes only one block of code at a time.
- Synchronous → The JS engine executes code sequentially, line by line.

Where does JS Runs ?

- Originally built only for browsers.
- Now can run into Servers by using Node.js
- JS runs inside JS Engine (V8 - Google, SipderMonkey - Firefox, etc.)

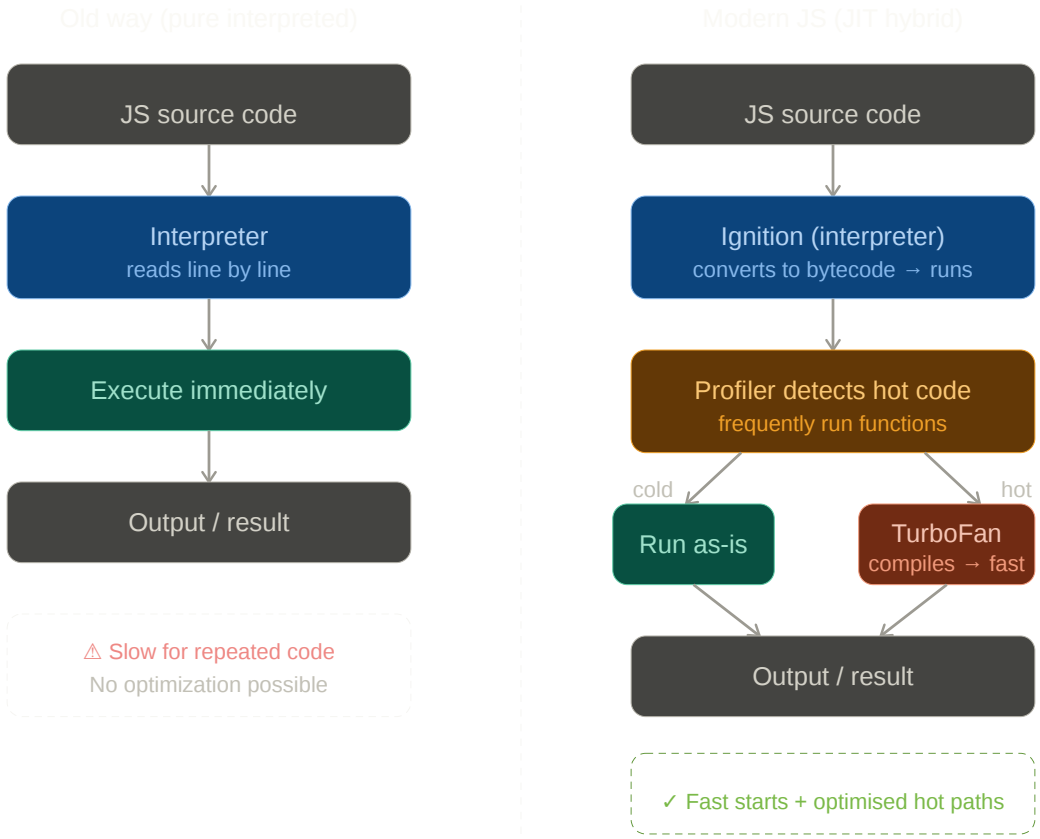
The Myth about JS is interpreted :-

In the past JavaScript was interpreted language which leads to performance limitations as code runs line by line. Over the time, as JS Engine advances JIT

(just-in-time compilation) have been introduced to improve performance.

- JIT(just-in-time compilation) → IJS engine first interprets the code to start execution immediately. As it runs, it identifies frequently used ("hot") code and compiles those parts into optimized machine code on the fly — giving the speed of compilation without the wait. No intermediate files are created.

> This is why modern JS is neither purely interpreted nor purely compiled — it's a hybrid, and this is what makes engines like V8 so fast.



JS Engines :-

A JavaScript engine is a computer program that executes JavaScript code and converts it into computer language.

Browser	Name of JavaScript Engine
Google Chrome	V8
Edge (Internet Explorer Old)	Chakra
Edge (New)	V8
Mozilla Firefox	Spider Monkey
Safari	JavaScript Core

How JS engines works internally (Pipeline) :-

- Step 1 — Loading the source the code into the memory. At this stage code is simply in plain text. The computer cannot understand it directly therefore engines must convert the code into a form that machines can executes.
- Step 2 — Lexical Analysis (Tokenization). The source is passed to a component called Lexer's (or Tokenizer). The Lexer's job is to break code into smaller pieces called tokens.

```
Example :

let x = 10;

becomes :

let
x
=
10
;
```

- Step 3 — Building the Abstract Syntax Tree (AST). AST is a tree-like representation of the program structure. AST acts as a blueprint of the program.

- Step 4 — Ignition generates Bytecode. Ignition converts the AST into Bytecode.
- **Step 5 — TurboFan optimizes hot code.** While bytecode runs, the **Profiler** watches for frequently executed ("hot") functions. TurboFan recompiles those into optimized machine code on the fly — this is the JIT step.
- Step 6 — Execution begins. Now the engine starts executing the bytecode. Before Execution JS create Global Execution Context.

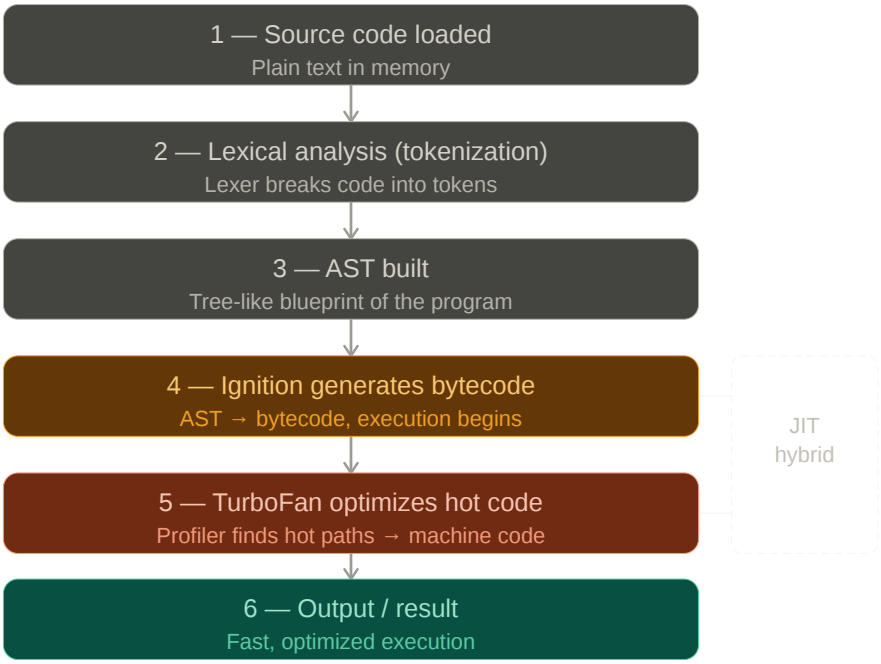
> This entire pipeline runs every time a JS file is loaded — in milliseconds.

JS engines by browser

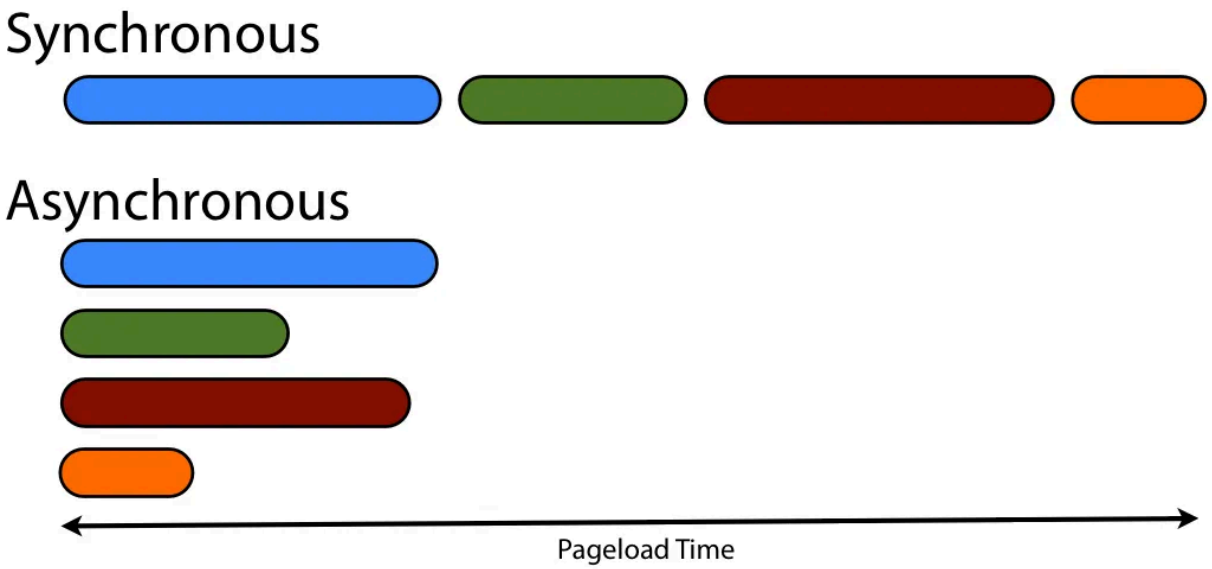
Browser	Engine
Google Chrome	V8
Microsoft Edge (new)	V8
Mozilla Firefox	SpiderMonkey
Safari	JavaScriptCore

Old IE / EdgeHTML used Chakra — replaced by V8 in 2020

How JS runs internally (V8 pipeline)



Single-Threaded & Synchronous model :-



What Single-Threaded means ?

> One commands process at a time.

JS code is read and gets executed line by line. Now JS is single-threaded, which means it has only one call stack (We will discuss Call stack and Execution Context in later topics).

What Synchronous means ?

> Step defined sequentially occur in the same order.

In simple words, synchronously means the code is executed a line after the other.

So for example, the code below is going to be executed as its written order

```
1 console.log("I'm gonna be executed first");
2 console.log("I'm gonna be executed second");
3 console.log("I'm gonna be executed third");
```

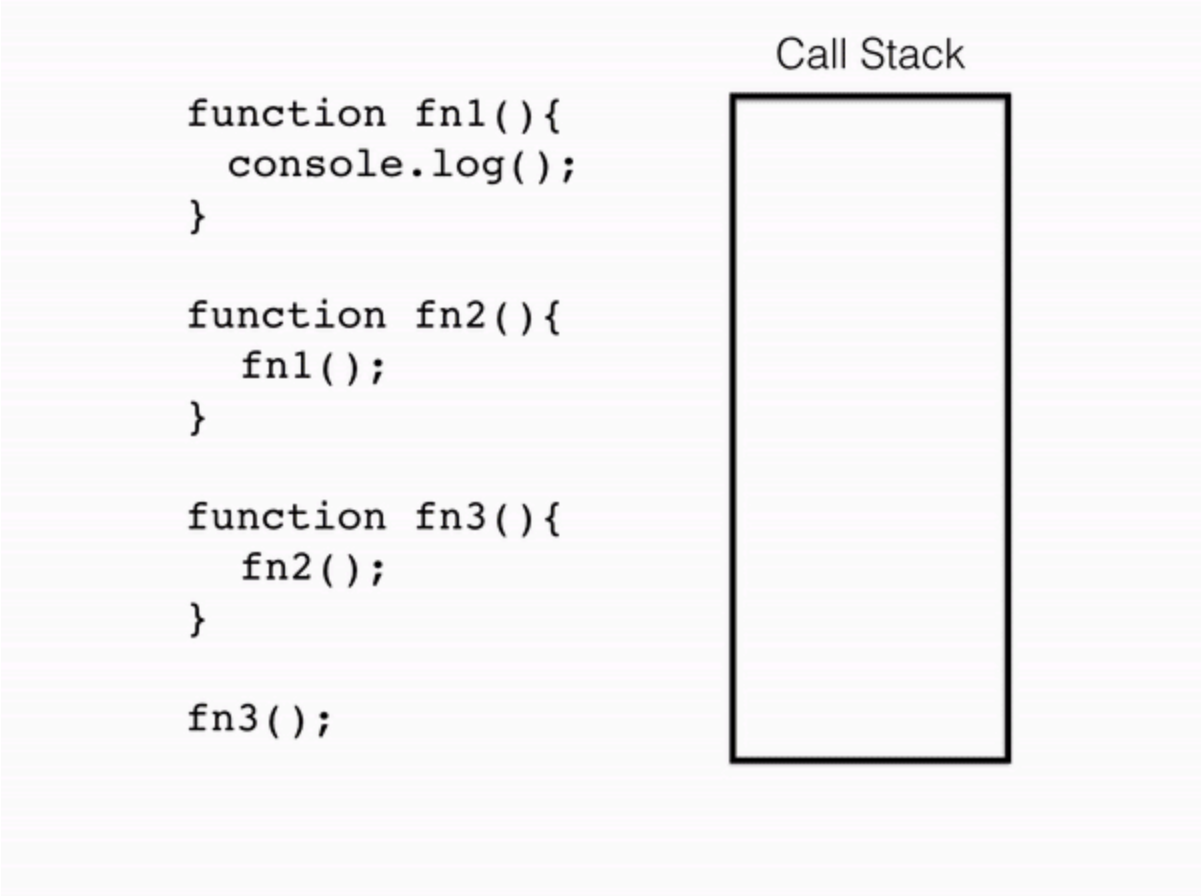
> Although JavaScript is single-threaded and synchronous by default, modern applications can perform asynchronous operations through the Runtime Environment, Web APIs, Callback Queue, Microtask Queue, and Event Loop.

Execution Context & Call Stack :-

Call Stack -

Call stack in JS is a structure that track the function calls. It operates on a basis ‘LIFO’ (Last in First out), meaning that last function called is the first to finish. As below image clear it more. Global Execution Context always sit at the bottom in the call stack.

One function after another comes in call stack so it is necessary to have base cases in functions rather Stack Overflow happens because the stack also has it memory and if we don’t give base case then infinite function calling done in recursion, because of this stack rans out of memory.



Execution Context -

An Execution Context is the environment in which JavaScript code is evaluated and executed. It contains information about variables, functions, the value of `this`, and references to the surrounding scope.

Imagine the place where all the code is getting executed is the Execution Context, which means the place where all the JS code runs is the place Execution Context.

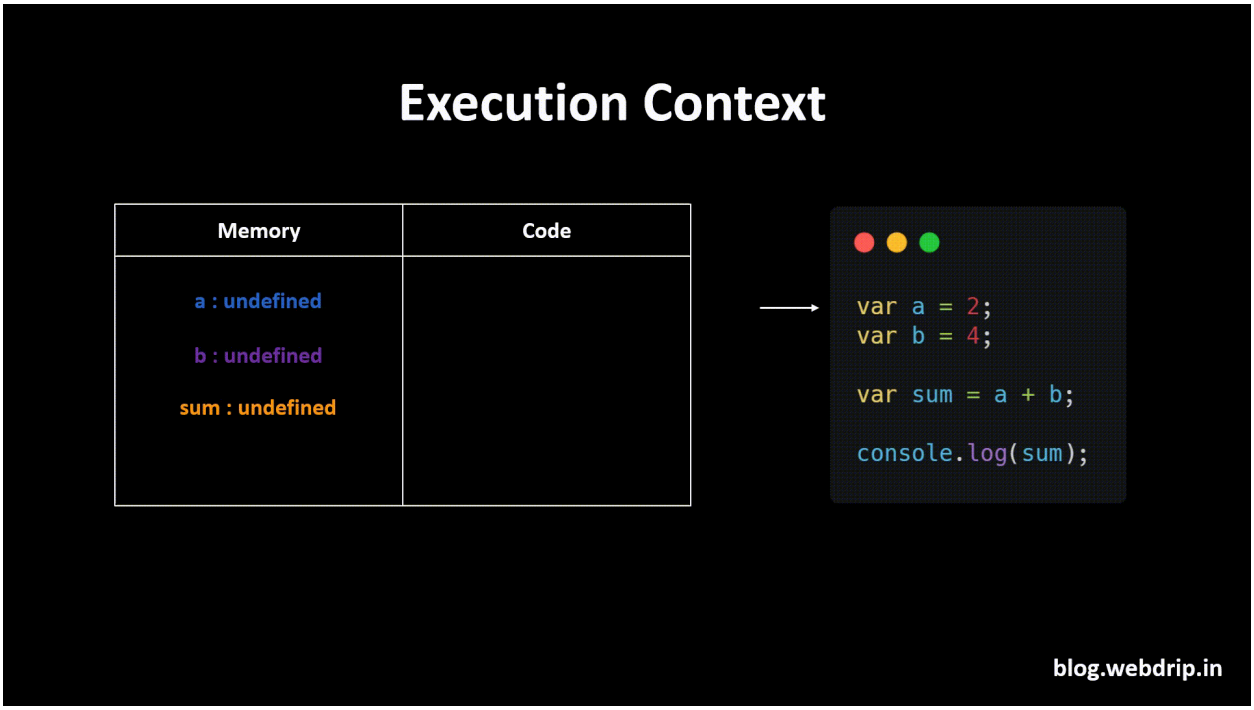
So when we run the JS code a Global execution context (GEC) is created. GEC consist of two phases a) Memory Creation Phase
b) Code Execution Phase

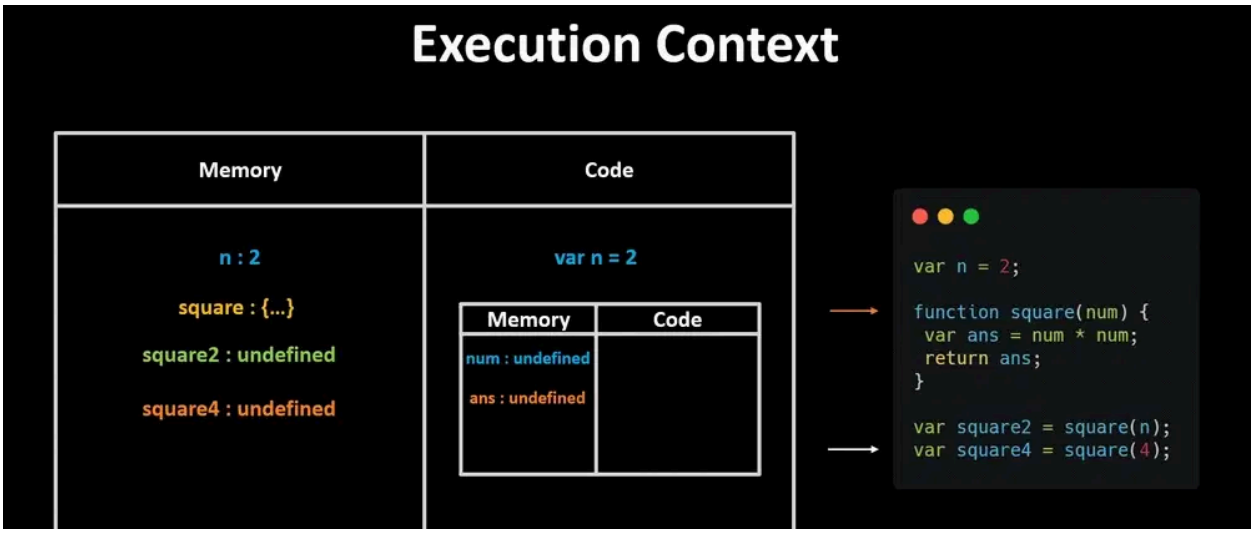
Phase 1 — Memory Creation Phase

- It allocates Memory for variables and functions, initializing variables with “Undefined” for Var, for let & const it is not initialized.
- In this phase the function is store with its body.

Phase 2 — Code Execution Phase

- Execute the code line by line after the memory allocation.
- Replace the “undefined” with actual value for variables during assignment.
- In this phase when function is called function make its own “Function Execution Context” in it.





Hoisting :-

Hoisting is JavaScript's behavior of allocating memory for declarations during the Creation Phase before code execution begins. No code is physically moved; the behavior is a result of how the JavaScript engine prepares memory.

As we see in the above section when GEC is created by JS then the var is created in memory phase and initialized as “undefined”. This is because the declared part of variables & functions are moved to the top of their containing scope during this phase 1

> *Only declarations are hoisted, not the initializations or assignments.*

Var Hoisting (Initialized as “undefined”) —

Variables declared using var are hoisted and initialized as “undefined”. So you can use them even they are not initialized like,

```
console.log(a) // You will get "undefined"
var a = 10;
```

```
console.log(a) // 10
```

let & const Hoisting (They are not initialized - The Temporal Dead Zone [TDZ]) —

let and const are also hoisted but they are not initialized as var. If we try to access them before their Declaration then we get reference error. Because let and const sits in Temporal Dead Zone (TDZ).

```
console.log(a) // Throw ReferenceError: Cannot access 'b' bef
ore initialization
let a = 10;
```

Scope :-

It is a region of a program where a variable can be accessed. In simple terms, scope determines the visibility and accessibility of variables within different parts of the code.

Global Scope —

The variables which is declared outside the function in global scope. They can be accessible everywhere, inside function, inside block.

```
let Name = "Guest"; // global scope
function greet() {
  console.log(`Hello ${Name}`); // We can use it
}
```

Function Scope —

The variables that are declared inside the function are only accessible inside the function. var, let & const all are function scoped.

```
function show(){
    let a = 10;
    var b = 20;
    const c = 30;
}
console.log(a, b, c) // ReferenceError — not visible outside
```

Blocked Scope —

Block is anything inside { } (if-else, for, while). let & const are blocked scoped. var ignores blocks.

```
if(true){
    let a = 10;
    var b = 20;
}

console.log(b); // 20
console.log(a); // // ReferenceError — not visible outside
```